

Senior AI/ML Engineer

Name: Ranjith Gopinath

Mail Id: ranjithgopinath878@gmail.com

Contact No: 562-336-6647

LinkedIn: <https://www.linkedin.com/in/ag-ranjith-6957011a2/>

Professional Summary:

- **AI/ML Engineer with nearly 7 years of professional experience** delivering Python-based applications, machine learning solutions, NLP pipelines, LLM applications, RAG systems, and production AI/ML services across financial services, healthcare/life sciences, and enterprise technology environments.
- Strong hands-on experience with **Python, PyTorch, TensorFlow, Scikit-learn, Pandas, and NumPy**, progressing from backend application development and data processing into enterprise **Machine Learning, NLP, and Generative AI** engineering.
- Developed production-oriented **Machine Learning** solutions covering data preparation, feature engineering, model training, validation, inference, model evaluation, retraining, and production troubleshooting using structured and unstructured datasets.
- Built practical **NLP** solutions for document classification, text processing, named entity recognition, information extraction, semantic similarity, document summarization, and enterprise search using transformer-based and traditional NLP techniques.
- Hands-on experience developing **LLM** applications using **OpenAI APIs, Hugging Face**, transformer models, prompt engineering, structured outputs, response validation, and application-level controls for enterprise use cases.
- Developed **RAG** applications covering document ingestion, preprocessing, chunking, embeddings, metadata handling, vector indexing, semantic retrieval, context assembly, and grounded LLM response generation.
- Worked with **LangChain** and **LangGraph** to create reusable LLM workflows involving retrieval chains, prompt templates, tool integration, workflow state, validation, and multi-step processing.
- Experience working with **AI Agents** and agent-oriented workflows using **LangGraph** and **CrewAI** for document analysis, information retrieval, summarization, validation, and controlled interaction with downstream services.
- Hands-on experience with **FAISS, Pinecone, Weaviate, and Azure AI Search**, tuning retrieval strategies, embeddings, chunking approaches, metadata filters, and similarity search for RAG applications.
- Implemented **MLOps and LLMops** practices using **MLflow**, experiment tracking, model versioning, evaluation workflows, artifact management, reproducibility, and controlled movement of ML/AI components across environments.
- Built and maintained production-facing AI/ML services using **Docker, REST APIs, Git, CI/CD** practices, and Python-based application components, supporting integration with existing enterprise applications and data platforms.
- Applied practical model and LLM evaluation methods including **precision, recall, F1-score**, validation datasets, retrieval relevance, response quality checks, hallucination checks, regression testing, and comparison of successive model or prompt changes.
- Earlier experience developing enterprise backend and full-stack applications using **Python, Django, Flask, FastAPI, React.js, REST APIs, SQL Server, and PostgreSQL**, providing a strong application engineering foundation for deploying ML solutions.
- Experienced in production support activities including log analysis, API troubleshooting, data validation, model inference investigation, defect resolution, root-cause analysis, release support, and coordination with application, data, QA, security, and business teams.
- Comfortable working in **Agile/Scrum** environments and contributing across requirements clarification, technical discussions, development, testing, code reviews, deployment, documentation, and production support while following enterprise coding and release practices.

Technical Skills:

Programming Languages	Python, SQL, JavaScript, JSON, XML
Machine Learning	Machine Learning, Scikit-learn, PyTorch, TensorFlow, feature engineering, model training, model validation, model evaluation, inference, hyperparameter tuning
NLP	NLP, text classification, Named Entity Recognition, text similarity, semantic search, text normalization, information extraction, document summarization, embeddings
Generative AI	LLMs, Generative AI, OpenAI APIs, GPT-4, Hugging Face, Transformers, prompt engineering, structured outputs, LLM evaluation, response validation
Vector Search	RAG, document ingestion, chunking, embeddings, semantic retrieval, FAISS, Pinecone, Weaviate, Azure AI Search, metadata filtering
AI Agents	LangChain, LangGraph, CrewAI, AI Agents, retrieval chains, tool calling, workflow orchestration
MLOps	MLflow, MLOps, LLMops, experiment tracking, model versioning, model evaluation, artifact management, model lifecycle, CI/CD
Data Processing	Pandas, NumPy, Python data processing, data validation, feature engineering, structured/unstructured data processing
Backend Development	Django, Flask, FastAPI, Django REST Framework, REST APIs, API integration

Frontend	React.js, JavaScript, HTML5, CSS3
Databases	SQL Server, PostgreSQL, SQLAlchemy, relational databases
Cloud / Platforms	Azure, Azure AI Search, cloud-based ML services
DevOps	Docker, Kubernetes, Git, GitHub Actions, Jenkins, CI/CD
Testing	PyTest, Postman, PyCharm, Jupyter, unit testing, integration testing
Methodology	Linux, Agile, Scrum, JIRA
Engineering Practices	Model monitoring, logging, exception handling, root-cause analysis, production support, code reviews, technical documentation

Professional Summary:

Client: Prudential Financial - Newark, NJ.

Dec 2025 – Present

Role: Sr. AI/ML Engineer

Responsibilities:

- Led the development of enterprise AI/ML solutions using **Python**, **PyTorch**, **TensorFlow**, **Scikit-learn**, and **Hugging Face** for financial-services use cases involving document intelligence, classification, entity extraction, prediction, and intelligent workflow automation.
- Developed production-grade **RAG** pipelines that combined enterprise documents, structured data, embeddings, and **LLMs** to provide context-aware responses for internal business and operational use cases.
- Built LLM application workflows using **LangChain**, **LangGraph**, and **OpenAI APIs**, including prompt templates, retrieval chains, tool calling, conversation memory, and response validation.
- Designed and implemented **AI Agents** using **LangGraph** and **CrewAI** to coordinate multi-step tasks such as document analysis, information retrieval, summarization, validation, and downstream system interaction.
- Integrated **GPT-4**, transformer-based models, and **Hugging Face** models into Python services while evaluating model behavior against accuracy, relevance, latency, and business requirements.
- Developed **NLP** pipelines for text classification, named-entity recognition, document summarization, semantic search, text normalization, and extraction of business information from unstructured content.
- Engineered document ingestion and preprocessing workflows using **Python**, OCR outputs, metadata extraction, chunking strategies, embeddings, and document-level indexing to support enterprise **RAG** applications.
- Implemented semantic retrieval using **vector databases** such as **FAISS**, **Pinecone**, and **Weaviate**, tuning chunking, embedding, metadata filtering, and similarity-search strategies for relevant context retrieval.
- Developed reusable ML and GenAI components in **PyCharm** using modular Python packages, configuration-driven pipelines, API integrations, logging, exception handling, and unit testing for production deployment.
- Applied prompt engineering techniques for structured outputs, grounding, summarization, classification, and reasoning workflows while incorporating validation checks to reduce irrelevant or unsupported model responses.
- Established **MLOps/LLMOps** workflows using **MLflow**, model/version tracking, experiment management, evaluation pipelines, artifact management, and controlled promotion of models and AI components across environments.
- Containerized AI/ML services using **Docker** and exposed model inference and RAG capabilities through **REST APIs**, enabling integration with enterprise applications and downstream business workflows.
- Built data preparation and feature-engineering pipelines using **Pandas**, **NumPy**, SQL, and Python-based processing to prepare structured and unstructured datasets for machine learning and LLM workloads.
- Developed model evaluation workflows covering precision, recall, F1-score, retrieval quality, hallucination checks, prompt-response validation, and regression testing for successive model and prompt changes.
- Implemented AI application controls around **PII protection**, input validation, access control, prompt filtering, sensitive-data handling, audit logging, and responsible use of enterprise data within LLM workflows.
- Collaborated with data engineers, application developers, architects, business analysts, and security teams to translate financial-domain requirements into deployable **AI/ML**, **NLP**, and **Generative AI** solutions.
- Troubleshooted model inference, retrieval, embedding, API, and data pipeline issues across development and production environments, analyzing logs and model outputs to identify root causes and improve application reliability.
- Contributed to technical design reviews covering **LLM architecture**, RAG patterns, model selection, vector search, agent orchestration, API integration, deployment strategy, and operational monitoring.
- Maintained technical documentation for model pipelines, RAG architectures, prompts, evaluation criteria, data flows, API contracts, deployment procedures, and production support processes.

Environment: Python, PyTorch, TensorFlow, Scikit-learn, Pandas, NumPy, NLP, LLMs, Generative AI, GPT-4, OpenAI APIs, Hugging Face, Transformers, RAG, LangChain, LangGraph, AI Agents, CrewAI, FAISS, Pinecone, Weaviate, MLflow, MLOps, LLMOps, Docker, REST APIs, SQL, Git, PyCharm, Jupyter, JSON, Linux.

Client: Pfizer – NYC, NY.

Feb 2025 – Nov 2025

Role: ML Engineer

Responsibilities:

- Developed and maintained machine learning solutions using **Python**, **Pandas**, **NumPy**, **Scikit-learn**, and **PyTorch** for structured and unstructured healthcare and life-sciences data.
- Prepared clinical, operational, and business datasets by handling missing values, inconsistent records, text normalization, feature engineering, data validation, and exploratory analysis.
- Built **NLP** pipelines for document classification, entity extraction, text similarity, keyword identification, and automated processing of large volumes of unstructured content.
- Applied **Natural Language Processing (NLP)** techniques including tokenization, embeddings, semantic similarity, text classification, named entity recognition, and document chunking for downstream ML applications.
- Developed and evaluated **LLM**-based solutions for extracting relevant information from business and healthcare documents while maintaining appropriate validation and response-quality checks.
- Worked with **LangChain** and **LangGraph** to develop reusable LLM workflows, retrieval pipelines, prompt chains, and controlled multi-step processing flows.
- Implemented **RAG** pipelines using document ingestion, chunking, embeddings, metadata filtering, vector search, retrieval, and grounded response generation.
- Integrated **OpenAI APIs** and **Hugging Face** models for NLP and generative AI use cases, including prompt development, model evaluation, and application-level integration.
- Worked with vector search technologies such as **FAISS**, **Pinecone**, and **Azure AI Search** to support semantic retrieval and knowledge-based applications.
- Developed reusable ML preprocessing, feature engineering, training, validation, and inference components to support consistent movement of models from experimentation into deployment.
- Used **MLflow** for experiment tracking, model versioning, artifact management, model evaluation, and maintaining reproducible machine learning workflows.
- Containerized ML applications and inference services using **Docker** and supported deployment through **Kubernetes**-based environments and CI/CD pipelines.
- Created automated model training and deployment workflows using **Git**, **GitHub Actions**, and cloud-based ML tooling to improve repeatability across development and production environments.
- Established model validation checks covering data quality, feature consistency, prediction behavior, model performance, and inference failures before production releases.
- Monitored deployed ML services and investigated model, data, API, and pipeline issues using application logs, monitoring dashboards, and operational alerts.
- Implemented **LLM evaluation** practices covering retrieval relevance, response quality, hallucination checks, prompt behavior, and groundedness for enterprise NLP applications.
- Applied prompt engineering techniques including system instructions, few-shot examples, structured prompts, output constraints, and context management to improve LLM application reliability.
- Added application-level **guardrails**, input validation, sensitive-data handling, and response filtering to support responsible use of NLP and generative AI solutions.
- Collaborated with data engineers and application teams to integrate ML models and NLP services with existing data pipelines, REST APIs, databases, and enterprise applications.
- Performed model troubleshooting and root-cause analysis for data drift, unexpected predictions, pipeline failures, inference errors, and production model behavior.
- Maintained technical documentation covering model workflows, feature definitions, data dependencies, experiment results, deployment procedures, and production support activities.
- Participated in code reviews, sprint planning, defect resolution, technical discussions, and release activities within an **Agile** development environment.
- Supported model retraining and enhancement activities by analyzing new data, reviewing prediction results, tuning model parameters, and comparing updated models against existing baselines.

Environment: Python, Pandas, NumPy, Scikit-learn, PyTorch, TensorFlow, NLP, LLMs, RAG, LangChain, LangGraph, OpenAI APIs, Hugging Face, FAISS, Pinecone, Azure AI Search, MLflow, Docker, Kubernetes, Git, GitHub Actions, REST APIs, Jupyter, SQL, Azure, CI/CD, MLOps, Generative AI.

Client: Honeywell - Charlotte, NC.

Jan 2023 – Nov 2023

Role: Python Full Stack Developer

Responsibilities:

- Created responsive user interfaces using **React.js**, **JavaScript**, **HTML5**, and **CSS3**, providing business users with screens for data entry, search, validation, status tracking, and record updates.
- Developed and enhanced enterprise web applications using **Python**, **Django**, **FastAPI**, **REST APIs**, **JavaScript**, **HTML5**, and **CSS3** to support internal business and operational workflows.

- Built reusable backend services in **Python** for processing equipment, service, operational, and transactional data received from internal applications and databases.
- Developed and maintained **RESTful APIs** using **FastAPI** and **Django REST Framework** to connect frontend modules with backend services and relational databases.
- Implemented business rules and server-side validations in **Python** to maintain data consistency across equipment records, workflow transactions, and operational requests.
- Integrated application services with **PostgreSQL** and **SQL Server**, writing SQL queries, joins, stored procedures, and data-access logic for application transactions and reporting requirements.
- Worked with existing enterprise APIs and internal services to exchange operational data between application modules and downstream systems.
- Added exception handling, logging, and input validation across backend services to simplify troubleshooting and prevent invalid transactions from reaching downstream processes.
- Developed search, filtering, pagination, and record-management functionality for large business datasets used by operations and support teams.
- Created unit and integration tests using **PyTest** and supported API validation through **Postman** to verify application behavior before deployment.
- Investigated production and UAT issues by reviewing application logs, database records, API responses, and user-reported scenarios, then implemented fixes through the normal release process.
- Used **Git** for source control and participated in code reviews, defect resolution, branch management, and controlled deployments across development, testing, and production environments.
- Supported CI/CD activities through **Jenkins**, including build validation, automated testing, and deployment of Python-based application components.
- Collaborated with business analysts, QA engineers, and other developers to clarify requirements, troubleshoot functional issues, and deliver enhancements within sprint timelines.

Environment: Python, Django, FastAPI, Django REST Framework, React.js, JavaScript, HTML5, CSS3, REST APIs, PostgreSQL, SQL Server, PyTest, Postman, Git, Jenkins, Linux, JSON,

Client: Cyient – Hyderabad, India.

Jul 2018 – Jul 2022

Role: Python Developer

Responsibilities:

- Wrote **SQL** queries, stored procedure calls, and data-validation routines to support application processing and reporting requirements.
- Developed application modules using **Python**, following existing project standards and coding practices for business and data-processing requirements.
- Created reusable Python functions and modules for file processing, data validation, business-rule checks, and application workflows.
- Developed and maintained **REST APIs** using **Flask** to support communication between internal applications and downstream services.
- Integrated application components with **SQL Server** and **PostgreSQL** databases using **SQLAlchemy** and Python database libraries.
- Implemented input validation, exception handling, logging, and error-management routines to improve application stability.
- Automated recurring operational tasks using **Python scripting**, reducing manual effort involved in file handling, data checks, and routine processing.
- Worked with **JSON**, **XML**, and CSV data formats for exchanging information between applications and external systems.
- Participated in requirement discussions with senior developers and business teams to understand functional changes and translate them into technical tasks.
- Fixed application defects identified during **SIT**, **UAT**, and production support, and performed root-cause analysis for recurring issues.
- Created unit test cases using **PyTest** and validated application changes before moving code through the release process.
- Used **Git** for source code management, branch updates, code reviews, and maintaining project changes across development environments.
- Supported application deployments across development, testing, and production environments and investigated issues reported after releases.
- Prepared technical documentation, deployment notes, troubleshooting steps, and application support information for ongoing project maintenance.
- Monitored application logs and batch-processing results, coordinated with dependent teams on failures, and followed issues through resolution.

Environment: Python, Flask, REST APIs, SQL, SQL Server, PostgreSQL, SQLAlchemy, PyTest, JSON, XML, CSV, Git, Linux, JIRA, Agile/Scrum.